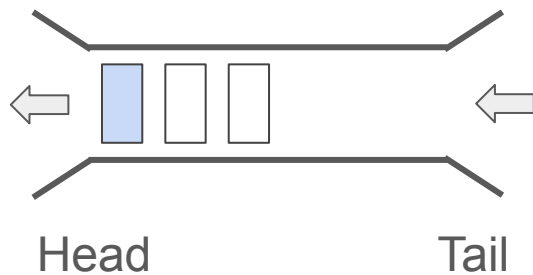


Data Structure (week 8)

講師：陳俊豪

Queue (佇列)：概念複習

- 主要對應現實中的「**排隊系統**」
 - 目的是為了**維持秩序**（**禁止插隊**）
 - 功能比 Array 強，但不比 Linked List
 - 核心精神：First-In-First-Out (**FIFO**)
 - 實體
- 專屬功能
 - Enqueue (入列) : $O(1)$
 - Dequeue (出列) : $O(1)$
 - Queue **Overflow** vs Underflow 問題
 - 透過動態擴容 (負載平衡) 解決 Overflow
 - 不需要特別處理 Underflow, 只要簡單檢查避免錯誤



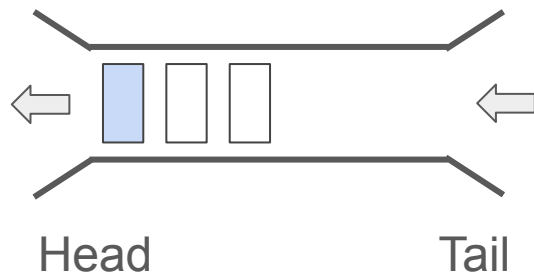
Queue (佇列)：概念複習

- 實際操作

- `from collections import deque` # 引入套件
- `queue = deque()` # 宣告
- `queue.append(10)` # Enqueue 正向 (右往左)
- `queue.popleft()` # Dequeue 正向 (右往左)
- `queue.appendleft(20)` # Enqueue 反向 (左往右)
- `queue.pop()` # Dequeue 反向 (左往右)

- 處理 Overflow vs Underflow

- Overflow：使用 if-else 去分配資料進到不同 Queue, 做到負載平衡
- Underflow：只需要簡單判斷 if length > 0 才能 pop 即可

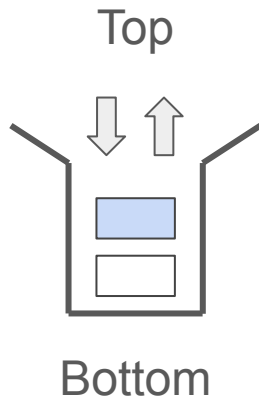


Stack

堆疊

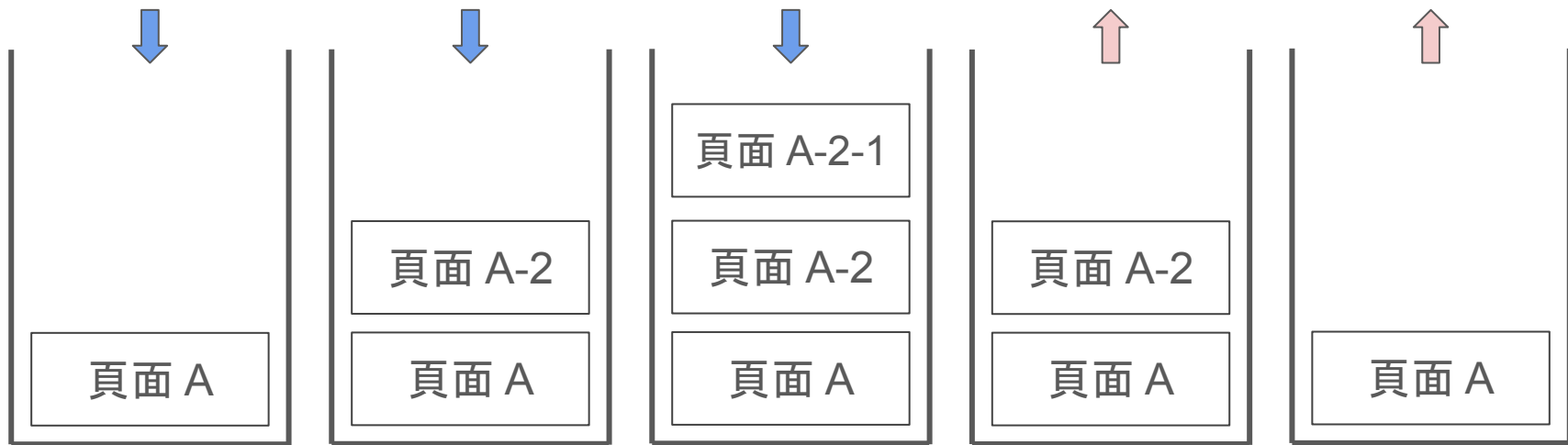
Stack (堆疊) :

- 主要對應現實中的「單一出入口」系統
 - 世界許多系統受限於「單一出入口」的物理限制，強迫必須後進先出！
 - 核心精神：Last-In-First-Out (LIFO)
 - 直覺：
 - p.s. 最後進來的，反而擁有最高優先權被處理
- 專屬功能 (Stack 的世界只有兩個合法動作)：
 - **Push** (入棧、推入)
 - 資料只能從「頂端 (Top)」加入 → **$O(1)$**
 - **Pop** (出棧、拿出)
 - 資料只能從「頂端 (Top)」離開 → **$O(1)$**
 - **Peek** (偷看最上面 Top) → **$O(1)$**



Stack (堆疊) :

- 用 Last-In-First-Out (LIFO) 想像一下瀏覽器「上一頁系統」的過程：
 - 逐步過程：進到頁面 A → 進到頁面 A-2 → 進到頁面 A-2-1 → 回到頁面 A-2 → 回到頁面 A



Stack (堆疊) : Python 實作

- 與 python list (Dynamic Array) 和 collections.deque (Linked List) 共用
 - 想想這兩個是不是都有 append 和 pop
 - Push → append()
 - Pop → pop()
- 那到底用誰？
 - **python list**
 - 直覺上會以為 push 未知數量的元素在固定記憶體下會導致搬家，進而提高時間複雜度
 - 但是，有沒有記得一件事 ...

Dynamic Array (動態陣列)

- Static Array 的缺點 (Numpy):
 - 固定大小，沒有彈性！
 - Insert 與 Delete 時，所有人都要 shift，時間複雜度高達 $O(N)$
- Dynamic Array：兩層架構，會搬家 > 有彈性
 - Step 1. 預留空間：一開始先跟作業系統要一塊連續的記憶體空間 (ex. 4 個位置)
 - Step 2. 正常使用：前 4 個資料直接放進去，瞬間完成 $O(1)$
 - Step 3. 需求過多：當需求超過空間時 (ex. 第 5 個)，去找一塊新的「更大」的連續空間
 - Step 4. 無情搬遷：將舊資料全部複製到新位置 (耗時 $O(N)$)，最後把舊記憶體註銷。
- 以上是 C/C++ 正常的運作過程，但是 ...

如果 append，那其實分攤後是 $O(1)$!!!

假設執行 N 次 append 且滿 2 的倍數要搬家 ex. 2 * 4 * 8 ...

第 1 次搬家：去找 2 個記憶體位置，搬了 1 筆資料
第 2 次搬家：去找 4 個記憶體位置，搬了 2 筆資料
第 3 次搬家：去找 8 個記憶體位置，搬了 4 筆資料
依此類推會是 $1 + 2 + 4 + \dots + N/2 = N - 1$ 必小於 N
總花費時間 = (一開始 N) + (搬家複製 $< N$) = 小於 $2N$

但如果是 Insert，那仍需要 $O(N)$ 算法解決，因為每次都會把後面的元素全部往右移



Stack (堆疊) : In-class exercises 1

- 回想上周的 Queue 是否有 Overflow 和 Underflow 問題, 那 Stack 會不會有 ?
 - Overflow ?
 - Underflow ?
- 題目 : 分散式平衡復原系統
 - 真實世界伺服器為了處理 Stack Overflow 問題, 配置了 3 個 Stack 緩衝區, 每個 Stack 的容量上限皆為 5 (注意 : 真實設計都是有上限的)。現在, 當使用者執行「存檔 (Push)」或「復原 (Pop)」時, 系統必須根據「負載平衡」原則來決定操作哪一個 Stack。
 - **PUSH 規則如下 :**
 - 以目前元素數量最少的 Stack 為主, 若數量相等則隨機分配。
 - 若 3 個 Stack 都滿了 (各 5 個), 則輸出 "Server Full", 同時強制終止程序 (表示當機)。

比賽分享 1：幸福科技 2026 (連結)



關於中心 各項資訊 關於廠商 學生創業 實務課程 吉林育成 進駐預約 創業健檢 2026技職盃黑客松競賽

2026幸福科技創新創業競賽

- 競賽簡介
- 申請辦法
- 經費補助
- 競賽時程
- 聯絡窗口與附件下載

首頁

2026幸福科技申請辦法

一、報名資訊

1. 參賽資格

- 團隊代表人須為**本校在校**生，以作為主要後續聯繫窗口與經費撥款負責人。
- 參賽團隊每組**2人以上，至多6人**，每人僅能參加1隊，可跨校組隊參加。
- 團隊可設指導老師至多2位，進行指導協助參賽。

2. 競賽報名：

- 於**115/06/15(一)**完成線上報名，報名成功要件為完整填寫團隊報名資料並上傳所有檔案。
- 上傳檔案：
 - 創意研發或是核心服務簡介說明簡報10頁（製作內容可參附件一，可自行設計與調整；「團隊名稱_幸福科技競賽簡報」）
 - 參賽聲明書（團隊全體成員皆須簽名後上傳）
 - 法定代理人同意書（未滿18歲須填寫法定代理人同意書）
- 報名網址：<https://forms.gle/m9PRu4unZrDrEk279>

- 鼓勵從生活問題或 SDGs (永續發展) 議題出發
- 每隊 2-6 人, 6/15 截止
- 第一階段會先選出 20 隊 (每隊 5,000 元材料費補助)
- 決賽在 11/5

比賽分享 2：中華電信智慧創新應用大賽校園組 2026

智慧創新應用大賽

競賽宗旨/徵件對象

競賽主題

評分標準

競賽獎金

競賽時程

競賽流程

文件下載

海地星空應用獎

注意事項

競賽時程

日期	項目
2026/9/18(五)12:00	報名截止
2026/10/16(五)	公布入圍決賽通知
2026/10/31 (六) (地點另行通知)	總決賽
2026/11/2(一)	公布入圍獲獎團隊
暫定2026/11/26(四) (地點另行通知)	頒獎典禮暨成果發表

- 智慧文化、製造、醫療、環境、交通、金融、**生活**、通信 八大主題
- 每隊 1-6 人, 初賽截止是 2026/9/18 中午 12:00
- 決賽是 10/31
- 期末先做靜態原型, 暑假慢慢補內容, 9 月再投

策略安排

- 人數抓 3~5 人一組
- 全班分成 10~15 組
- 先參加「幸福科技 2026」, Finetune 後再參加「中華電信」
- 年底再參加資財系的「資財之星」
- 策略：
 - **不要做 不要做 不要做** 大型系統, 而是主打「用 AI 幫人做生活決策」的作品
 - **故事最為重要！故事最為重要！故事最為重要！**
 - 報告人台風很重要, 準備 Q&A 也很重要
 - 有 **Demo** 有加分 (靜態網站)

比賽題目範例：Zero Day - 零日倒數 Budget Tree

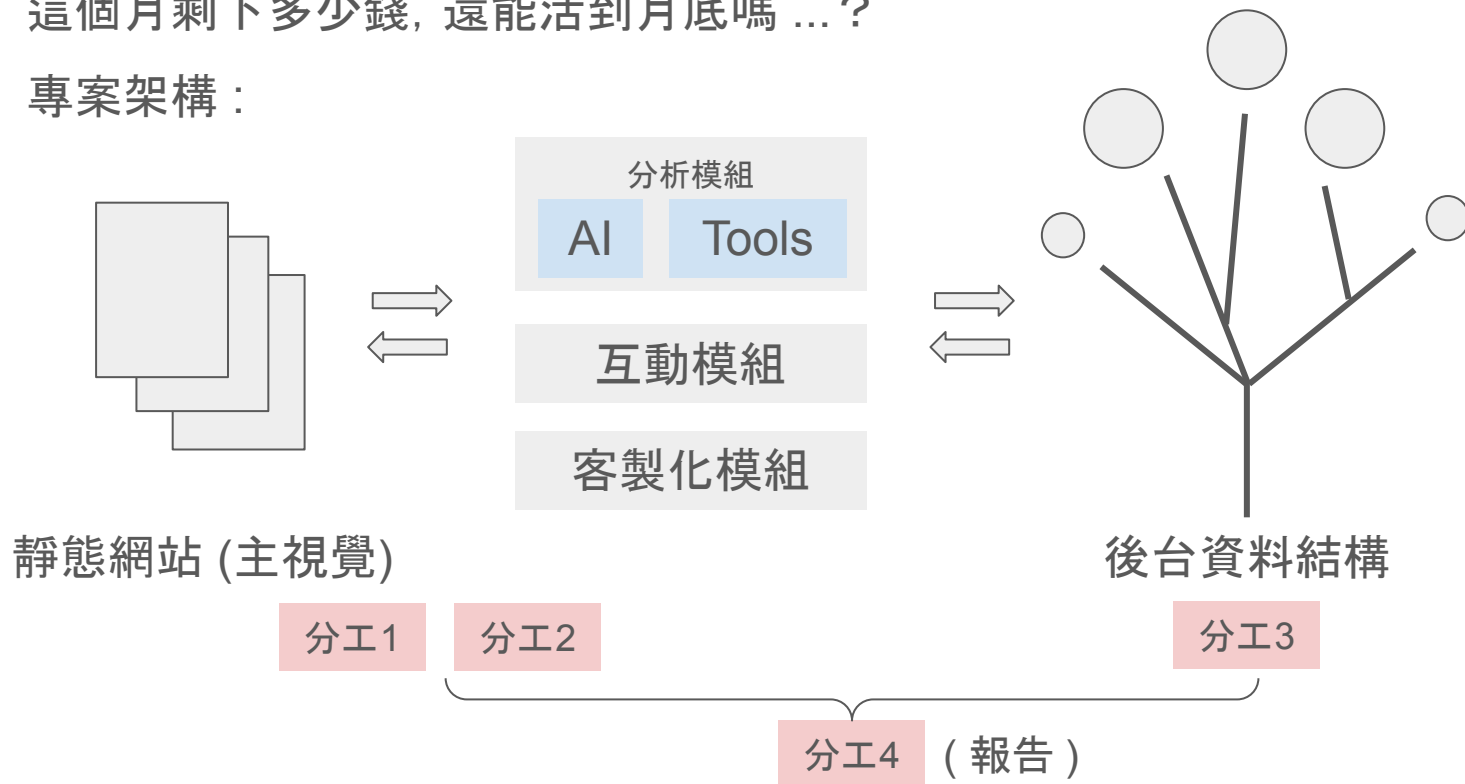
- 這個月剩下多少錢，還能活到月底嗎 ...？
- 情境說明：
 - 這個月生活費 4,500 元，現在只剩 1,280 元。接下來 8 天還有：午餐、晚餐、影印報告、社團聚餐、交通儲值、可能要買書，而且還有很多東西想買，發瘋！
 - 網站首頁是一棵預算樹：「必要支出」、「可調整支出」、「可延後支出」三大枝幹往下展開。
 - 使用者按分析按鈕後，系統會告訴他：
 - 這個月最花錢的是飲料
 - 規劃生存策略 A：如果把 5 次手搖改成 2 次，你可以多活 6 天
 - 規劃生存策略 B：如果教材改買二手書，你可以再省 350 元
 - 補充：你現在最不該砍的是交通與影印，最該先砍的是衝動性消費（ex. 零食）
 - 網頁內容：首頁（樹）、調整模擬器、生存建議頁，做手機版

比賽題目範例：Zero Day - 零日倒數 Budget Tree

- 這個月剩下多少錢, 還能活到月底嗎 ... ?
- 技術設定：
 - 主資料結構：Tree
 - 其他結構：
 - Tool A：Hash Table 做及時資料分配
 - Tool B：Sorting 負責排出「最花錢前五名」「最值得先砍的項目」
 - Tool C：Searching 讓使用者找特定日期、類別或高額異常支出
 - 核心判斷 AI：
 - Gemma 4 免費 Gemini API Key

比賽題目範例：Zero Day - 零日倒數 Budget Tree

- 這個月剩下多少錢，還能活到月底嗎 ...？
- 專案架構：

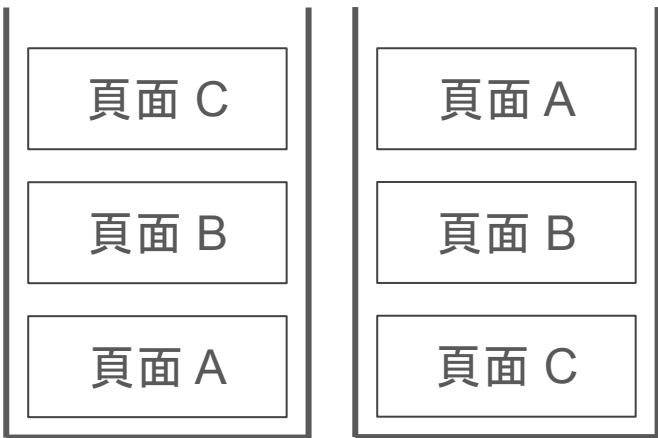


比賽題目範例：Zero Day - 零日倒數 Budget Tree



Two-Stack 演算法

- 有兩個 Stacks 的目的是什麼？
 - 主要目的：**重新排列 (改變順序)**
 - 單個 Stack 是 LIFO, 兩個 Stacks 就能變回 FIFO (Queue)



Two-Stack 演算法：思考題

- 現在有兩個空 Stacks, 分別叫 S1 和 S2。我們要把字母放進去：
 - 放入 (PUSH X): 把字母 X 丟進 S1。
 - 倒過來 (FLIP): 把 S1 的東西「全部」一個一個搬到 S2。
 - 拿出來 (GET): 從 S2 拿出(pop)最上面的一個。
- 請判斷以下得到的結果？
 - PUSH A
 - PUSH B
 - PUSH C
 - FLIP (把 S1 的東西全倒進 S2)
 - PUSH D
 - GET (從 S2 拿出一個)
 - GET (再從 S2 拿出一個)

Two-Stack 演算法：In-class exercises 2

- 題目情境：智能物流中心
- 物流中心有三個區域：
 - S1 (進貨區 - Stack): 進貨時，貨物會一箱一箱疊上去。
 - S2 (分類區 - Stack): 為了讓最早進貨的貨物先出貨，我們需要把 S1 的貨物倒過來放進 S2。
 - Q (貨車 - Queue): 最終出貨的順序，必須是先進先出 (FIFO)。
- 操作規則：
 - ARRIVE(item): 貨物到達，直接 push 進 S1。
 - SHIP(): 執行出貨動作。
 - 規則 A: 如果 S2 是空的，必須先把 S1 的所有貨物「一個一個」移到 S2。
 - 規則 B: 只要 S2 裡面有東西，直接從 S2 拿出最上面的貨物，放進貨車 Q。
 - 規則 C: 如果 S1 和 S2 都是空的，輸出 "Warehouse Empty"。

AI Tools : Gemini 3.1 Pro (Free)

samsa, 你好

你想從哪裡著手？

問問 Gemini

+ 工具

Pro



 生成圖片

 創作音樂

撰寫任何內容

幫助我學習

提升

Gemini 3

快捷

回覆快速

思考型

解決複雜問題

Pro

3.1 Pro 可處理進階數學和程式設計

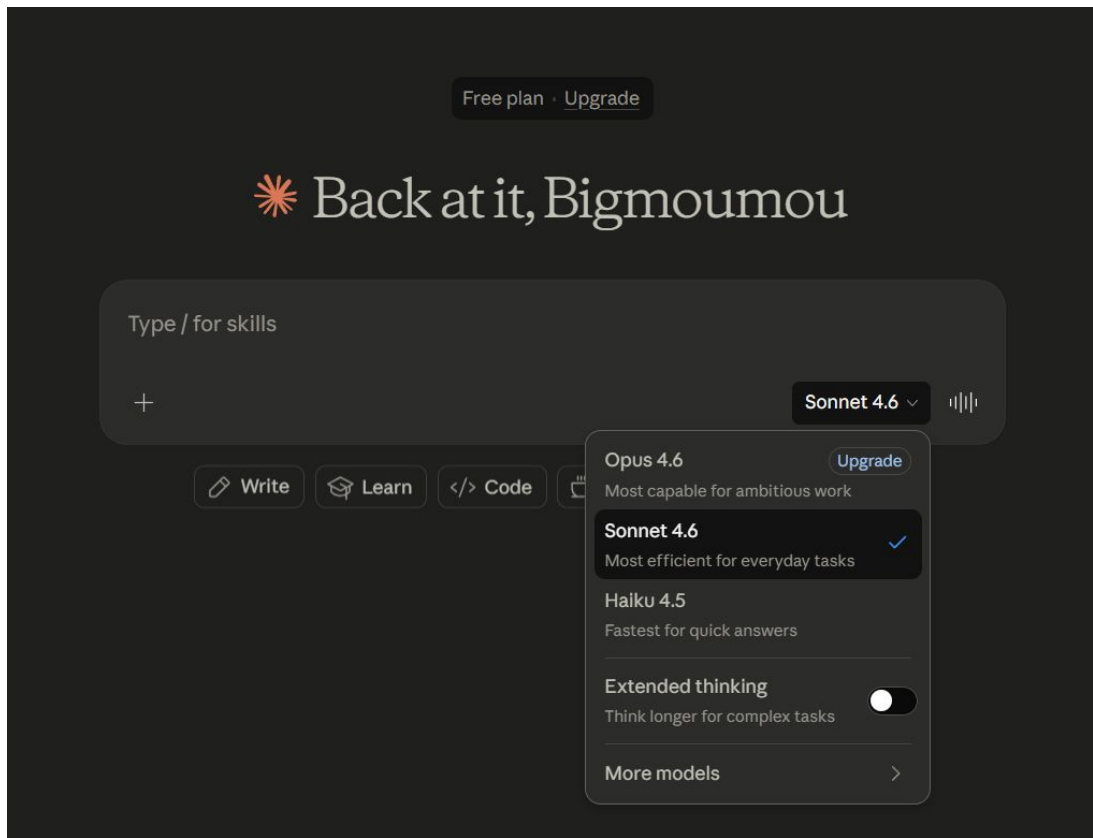


Google AI Plus

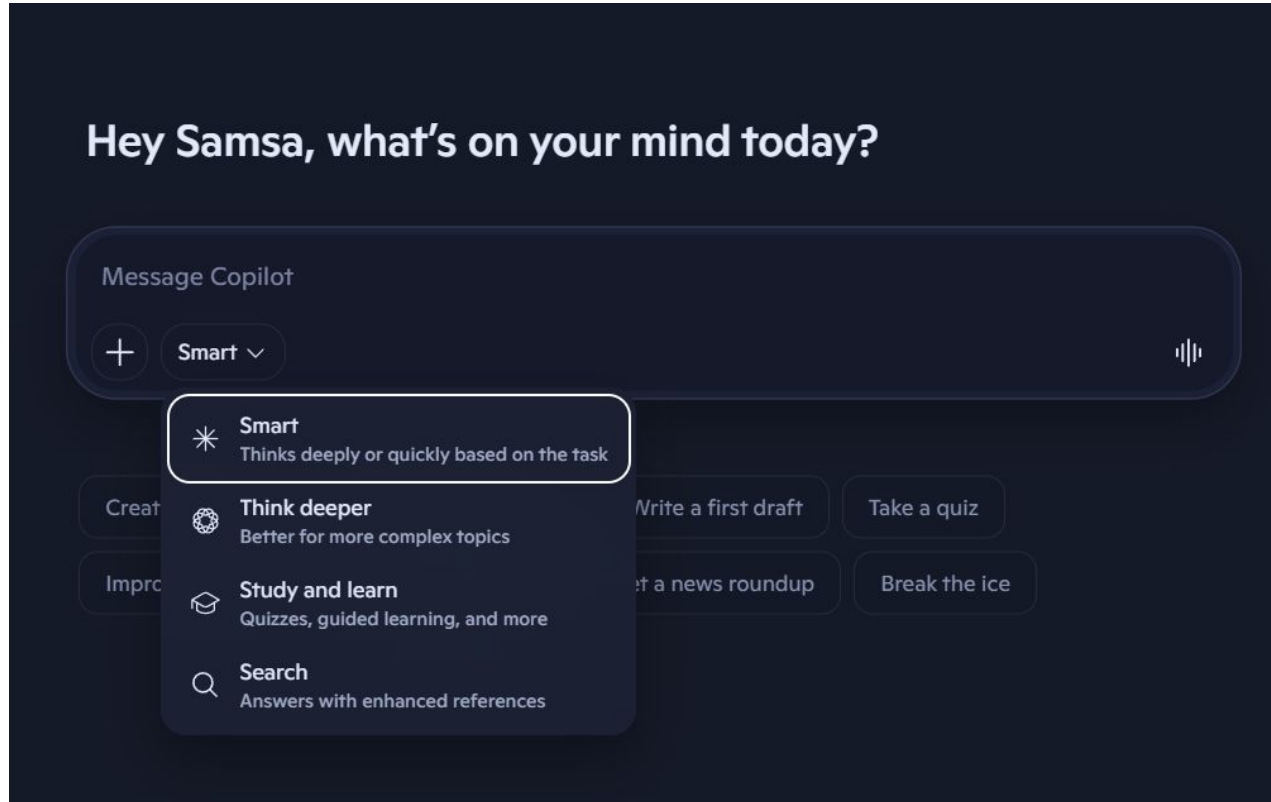
享有更多模型和功能使用權限

升級

AI Tools : Claude Sonnet 4.6 (Free)



AI Tools : Microsoft Copilot (Free)



期中考 (40%) 需知

- 提交檔案必須為 py 或 ipynb 檔, 不能只給連結
- 可以使用 AI, 需自己準備 (任意 AI)
- 至少會出 4 題
 - 一樣只會給測資 A, 但實際老師會用更多測資去測
- 上課時間內寫完 (考到 3:30), 寫完後把檔案寄到指定信箱
 - 會認寄信時間, 超過不收件
 - 和 HW 不同不會現場改
- 沒有回答問題的解釋題
- 會有送分題